

ShowUI- π : Flow-based Generative Models as GUI Dexterous Hands

Siyuan Hu* Kevin Qinghong Lin* Mike Zheng Shou[†]

Show Lab, National University of Singapore

<https://showlab.github.io/showui-pi>

Abstract

Building intelligent agents capable of dexterous manipulation is essential for achieving human-like automation in both robotics and digital environments. However, existing GUI agents rely on discrete click predictions (x, y) , which prohibits free-form, closed-loop trajectories (e.g., dragging a progress bar) that require continuous, on-the-fly perception and adjustment. In this work, we develop ShowUI- π , the first flow-based generative model as GUI dexterous hand, featuring the following designs: (i) **Unified Discrete-Continuous Actions**, integrating discrete clicks and continuous drags within a shared model, enabling flexible adaptation across diverse interaction modes; (ii) **Flow-based Action Generation** for drag modeling, which predicts incremental cursor adjustments from continuous visual observations via a lightweight action expert, ensuring smooth and stable trajectories; (iii) **Drag Training data and Benchmark**, where we manually collect and synthesize 20K drag trajectories across five domains (e.g., PowerPoint, Adobe Premiere Pro), and introduce ScreenDrag, a benchmark with comprehensive online and offline evaluation protocols for assessing GUI agents' drag capabilities. Our experiments show that proprietary GUI agents still struggle on ScreenDrag (e.g., Operator scores 13.27, and the best Gemini-2.5-CUA reaches 22.18). In contrast, ShowUI- π achieves 26.98 with **only 450M parameters**, underscoring both the difficulty of the task and the effectiveness of our approach. We hope this work advances GUI agents toward human-like dexterous control in digital world.

1. Introduction

Building intelligent assistants capable of dexterous manipulation is essential for achieving human-like automation in both physical and digital environments [23, 26, 47]. For example, in the physical world, robotic dexterous hands are deployed for manipulation tasks e.g., sorting objects on physical desktops [3, 4, 36, 57]. Analogously, in the digital

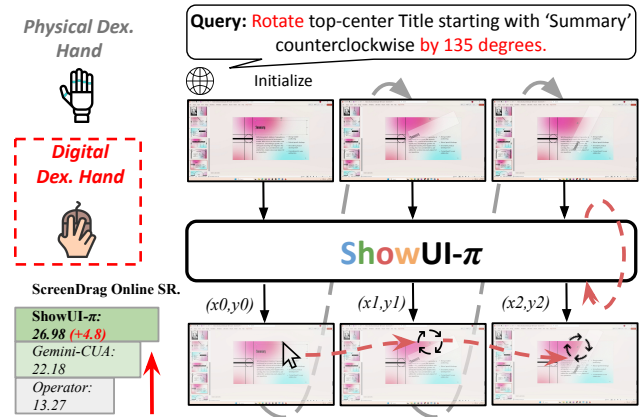


Figure 1. ShowUI- π is a lightweight flow-based generative model for GUI Automation that handles dragging actions requiring on-the-fly observation, such as drawing and Captcha solving. Given a query, ShowUI- π efficiently generates corresponding continuous trajectory from streaming visual observations.

world, Graphical User Interfaces (GUI) serve as the primary medium through which people interact with computers, and automating GUI interactions holds substantial promise for enhancing productivity [11], reducing workload [49], and improving accessibility [10].

Recent advances in large language models (LLMs) [1, 13, 19, 33, 55] and vision-language models (VLMs) [8, 18, 21, 22, 41] have accelerated progress in GUI agents [6, 26, 31]. These emerging visual-centric agents directly perceive screen observations and output actions such as clicking or typing. However, most existing GUI agents [42, 46] are obtained by fine-tuning foundation VLMs without architecture adaptation, which represent action's coordinate in a discrete, tokenized form. This representation fundamentally restricts agents from executing complex high-degree-of-freedom dragging, such as creative drawing or Captcha-solving by rotation *i.e.*, tasks inherently demanding continuous, real-time visual observation and responsible for incremental trajectory adjustment. By contrast, Vision-Language-Action models [3, 9, 36] in robotics leverage flow-based generative methods (e.g., diffusion policy, flow matching) to enable achieving such continuous, fine-

*Equal contribution.

[†]Corresponding author.

grained control on the fly.

Motivated by how humans control the mouse for fine-grained cursor movement, *i.e.*, continuously perceive and adjust actions, we wonder: *can we construct such a digital dexterous hand in GUI?* We propose ShowUI- π , the first flow-based GUI model designed for continuous trajectory. Specifically, ShowUI- π highlights the following architecture: **(i) Unified Discrete-Continuous Actions:** ShowUI- π casts discrete clicks as drags with negligible movements, and integrates them with continuous drags into a unified modeling. Under this formulation, both action types are represented by a sequence of (x, y, m) triplets, where (x, y) are cursor coordinates and $m \in \{\text{down}, \text{up}\}$ is the mouse button state. This unified design allows ShowUI- π to handle both drag and click tasks with a single shared model, adapting without task-specific head selection. **(ii) Flow-based Action Generation:** Different from existing GUI agents predicting discrete, tokenized actions from language decoding, *e.g.*, `click(x, y)` and `drag(start, end)`, ShowUI- π is a flow-based generative model, and employs a lightweight action expert to incrementally predict cursor adjustments from continuous visual observations. Built on a transformer backbone, the action expert is trained with flow matching to generate stable and precise action trajectories; **(iii) ScreenDrag benchmark:** We construct a benchmark specially for continuous GUI tasks, including 505 real-world drag tasks across five domains covering both professional control and daily usage, such as PowerPoint, OS Desktop and file manager, Handwriting on canvas, Adobe Premiere Pro, and Captcha solving, with 101 tasks for each domain. Moreover, to fully evaluate continuous trajectories, we introduce two complementary modes: an offline open-loop evaluation using average trajectory error and endpoint accuracy, and an online closed-loop evaluation using task success rate. **(iv) Continuous Trajectory Training Data:** To advance the model training, we construct a training dataset of 20K manually collected and synthesized drag trajectories across the five aforementioned domains and 11 categories of tasks, and all trajectories have recorded UI states and dense coordinates.

Our experiments show that proprietary GUI agents still struggle on ScreenDrag (*e.g.*, Operator scores 13.27, and the best Gemini-2.5-CUA reaches 22.18). In contrast, ShowUI- π achieves 26.98 with only 450M parameters, underscoring both the difficulty of the task and the effectiveness of our approach. Moreover, our ablation studies further reveal the superficial nature of standard flow-matching training and highlight the substantial impact of individual designs. Our contributions are three-fold:

- **The first flow-based GUI agent for continuous trajectories.** To the best of our knowledge, we are *the first work* to tackle continuous trajectory-based drags in GUI automation, revealing core limitations of discrete, tok-

enized actions. We propose ShowUI- π , a lightweight 450M flow-based VLA that unifies discrete clicks and continuous drags within a shared modeling.

- **ScreenDrag training dataset and benchmark suite.** We introduce ScreenDrag, a benchmark tailored for continuous GUI manipulation, with 505 real-world drag tasks across five domains and 20K manually collected and synthesized drag trajectories over 11 task categories. ScreenDrag supports both offline open-loop metrics and online closed-loop success evaluation.
- **Comprehensive evaluation and key insights.** Experiments show that proprietary GUI agents still struggle on ScreenDrag (*e.g.*, Operator 13.27, best Gemini-2.5-CUA 22.18), while ShowUI- π reaches 26.98 with only 450M parameters, highlighting both task difficulty and model effectiveness. Ablations further demonstrate the impact of our individual design choices for achieving robust continuous control.

2. Related Work

2.1. Digital GUI Automation.

LLM-based agents in digital environments have evolved from reasoning and tool-use paradigms, such as Chain-of-Thought [43], ReAct [50], and related prompting frameworks [15, 48, 52], toward task-oriented GUI automation [16, 25]. Existing methods include *training-free* pipelines which plan with a VLM and execute via external tools [14, 54], and *training-based* models jointly learning perception and action from screenshots and instructions [8, 12, 21, 51]. Existing GUI agents, including ShowUI, decode actions into discrete text tokens, simplifying integration with VLM planners but limiting control to simple clicks or short drags [24]. ShowUI- π departs from these methods by directly modeling continuous spatial trajectories for long, smooth, temporally coherent manipulations.

2.2. Physical Robotic Manipulation.

Generalist Vision-Language-Action (VLA) models map natural-language instructions and visual observations to action policies by leveraging large multimodal backbones and robotics data at scale. Representative lines include RT-1 [4] and successors [57] which transfer knowledge from web-scale vision-language pretraining to robot control, open-source VLA backbones and compact variants such as OpenVLA [23], PaliGemma [2], and TinyVLA [44], as well as recent works enriching training signals with visual traces and language alignment [32].

Discrete Language Models formulations predict low-level controls from decoded discrete action tokens. Representative models include RT-2 [57] and OpenVLA [23], which discretize continuous actions into 256 bins per dimension and overwrite the least-used tokens in the language model

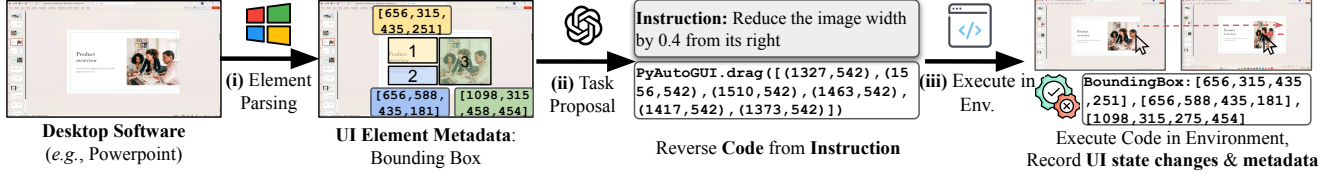


Figure 2. **ScreenDrag Automated Data Collection Pipeline.** ScreenDrag automated data generation pipeline for continuous trajectory-based GUI interaction data. The pipeline includes three stages: **(i) Element Parsing:** The software application UI is parsed with UI Automation of Windows SDK in order to retrieve the UI element metadata, *i.e.*, bounding box, *etc.* **(ii) Task Proposal:** Given the UI element metadata, an LLM will be prompted to generate a drag instruction, *i.e.*, *reduce the image width by 0.4 from its right*, and the expected metadata change. Afterwards, the corresponding drag code with dense trajectory will be generated. **(iii) Trajectory Synthesis:** The drag code will be executed in the software environment. A rule-based verifier will check the parsed metadata from UI states before and after the drag to ensure that the metadata change satisfies the expectation.

vocabulary to represent these action bins as autoregressively generated tokens; and Magma [47], which applies discrete action tokenization across both UI navigation and robotic manipulation tasks. Language-modeling maintains compatibility with instruction-tuned VLMs but suffers from temporal quantization when fine motor control is needed [35].

Continuous Generative Models parameterize a time-indexed transport from a base distribution to action trajectories. Within this paradigm, (i) *Diffusion Policies* instantiate the transport via score-based denoising, demonstrating strong capabilities in visuomotor control due to stable optimization and high-dimensional handling [9, 20, 40, 56]. Alternative frameworks such as (ii) *Flow Matching* and *Rectified Flow* directly regress a time-conditioned velocity field along a predefined probability path, eliminating explicit score estimation and iterative denoising for deterministic ODE-based sampling. Rectification further simplifies trajectory modeling and enhances sample efficiency [27, 28]. Such properties have inspired their integration into robotics-oriented flow-matching VLA policies, combining a flow-based action head with VLM backbones for smooth, real-time continuous control [3, 36]. The learning objective is:

$$\mathcal{L}_{\text{flow matching}} = \frac{1}{T} \sum_{t=1}^T \|\mathbf{v}_{\theta}(\hat{\mathbf{a}}_t, t \mid o_t, \mathcal{Q}) - \mathbf{u}_t\|^2, \quad (1)$$

where $\mathbf{v}_{\theta}(\hat{\mathbf{a}}_t, t \mid o_t, \mathcal{Q})$ denotes the predicted velocity field at time t conditioned on the current observation o_t and task instruction $\mathcal{Q}$, $\mathbf{u}_t$ is the target velocity along the probability path, $\hat{\mathbf{a}}_t$ is the intermediate action state, and T is the number of training timesteps.

3. ScreenDrag

3.1. Problem Definitions

Unlike discrete actions such as clicking, which can be completed with one command based on an initial observation, continuous actions—such as dragging or rotation—require on-the-fly visually grounded trajectories in real time. Formally, given an initial observation o_0 and a language instruction $\mathcal{Q}$, a GUI agent policy π sequentially predicts a continuous trajectory $\tau = \{a_t\}_{t=0}^T$, where each action $a_t = [x, y]$ denotes the spatial coordinates at time step t .

However, existing benchmarks reduce drag-like interactions to discrete start–end point pairs and provide only a single screenshot per atomic action, ignoring the rich intermediate state changes that occur during continuous dragging. In addition, many drag scenarios allow multiple valid trajectories to complete a task, yet this flexibility is not captured in current formulations. To address these limitations, we introduce ScreenDrag, a benchmark designed to evaluate GUI agents’ capabilities in performing continuous actions.

3.2. Dataset Construction

Data Source. ScreenDrag comprises a diverse set of real-world drag tasks including (1) PowerPoint element manipulation such as rotation, (2) File drag sorting in OS desktop and File manager, (3) Handwriting on canvas, (4) Premiere Pro asset manipulation between workbench and tracks, as well as (5) Captcha-solving. These tasks cover both daily usage and professional control, requiring precise and continuous spatial control, and observation on-the-fly during the task execution.

Data Curation. Notably, high-quality drag data must capture screen state changes, dense cursor trajectories, and diverse task instructions. A straightforward approach is to manually record human demonstrations, but this is labor-intensive, thus not scalable. To overcome this limitation, we propose a scalable data collection pipeline that leverages automated drag execution, as illustrated in Fig. 2. Our data curation approach includes three stages: **(i) Element Parsing:** We first extract the element bounding box metadata using UI Automation of Windows SDK [53], to collect a set of candidate elements to be dragged, *e.g.*, an image in a PowerPoint slides. **(ii) Task Proposal:** Then we use an LLM (Qwen-2.5-72B [37]) to propose candidate instructions, *e.g.*, *reduce the image width by 0.4 from its right*. **(iii) Trajectory Synthesis:** Next, we synthesize drag PyAutoGUI code containing dense trajectory coordinates and sending it to a code executor, which performs the drag actions on the real operating system and software applications. To complement this automated process, we also collect human demonstrations of drag interactions. Each human demonstration includes a high-resolution 60 FPS

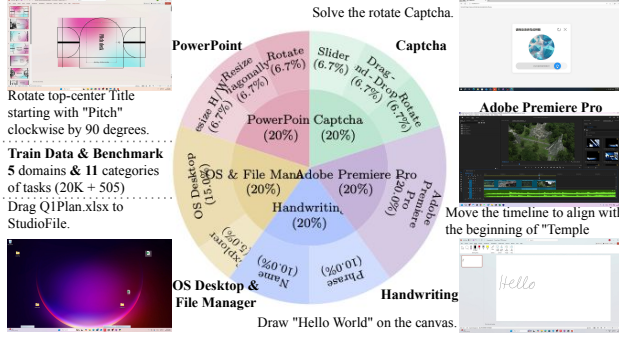


Figure 3. **ScreenDrag Data Distribution.** The inner ring indicates the five equally distributed domains. The outer ring demonstrates per-category breakdowns with shares of the full dataset.

screen recording that captures UI state changes, the executed dense trajectory, and the corresponding instruction.

Data Analysis. As shown in Fig. 3, ScreenDrag comprises a training dataset of 20K manually collected and synthesized drag trajectories, with high resolution screen recording and metadata of the UI changes, as well as dense trajectories during the drag. The training dataset spans across five aforementioned domains and 11 categories of tasks. Moreover, ScreenDrag includes a benchmark of 505 trajectories for evaluation (101 tasks for each domain). The average duration of screen recordings is 9.62 seconds, and the average number of frames is 577 frames per recording. Refer to the Supplementary Material for more detailed analysis.

3.3. Evaluation Metrics

Besides preparing training data, an effective evaluation protocol for model-generated drag trajectories is also essential. As noted earlier, the benchmark must expose UI state changes to the model throughout the drag process. Moreover, evaluation should go beyond assessing a simple start-end linear drag and instead offer a dynamic environment in which the model must perform drag actions incrementally. To this end, ScreenDrag introduces two complementary evaluation modes for drag tasks: offline evaluation and online evaluation.

Offline Evaluation is performed as an open-loop [45] assessment of the policy, measuring the policy’s ability to replicate the ground-truth stepwise behavior without compounding errors. Given a static environment where screenshot, task and oracle previous state are provided, offline evaluation assesses whether the policy can successfully reach the next trajectory point, and the amount of the error, which can be represented by two metrics:

(i) *Average Trajectory Error* evaluates the mean square error of the entire trajectory (aligned with Eq.1), defined as the average Euclidean distance between predicted and ground-truth coordinates across all timesteps:

$$ATE = \frac{1}{T} \sum_{t=0}^T \|\hat{\mathbf{a}}_t - \mathbf{a}_t^{gt}\|_2 \quad (2)$$

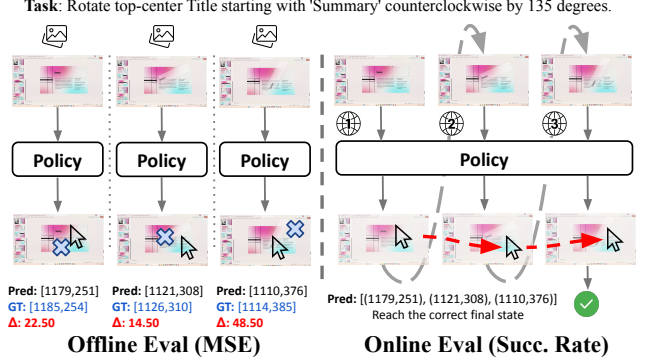


Figure 4. **Comparison between offline evaluation and online evaluation pipelines of ScreenDrag:** (i) **Offline evaluation** is based on the distance of prediction and ground-truth in independent trunks; (ii) **Online evaluation** is incremental over sequential trunks, and based on the final outcome.

providing a detailed assessment of the spatial accuracy and smoothness of interactions step by step throughout trajectory execution.

(ii) *Trajectory Endpoint Accuracy* emphasizes endpoint precision, measuring the proportion of predicted trajectories whose endpoints fall within a predefined spatial tolerance radius ϵ , e.g., 20 pixels from the ground-truth endpoints. This metric highlights the agent’s ability to accurately reach the intended final interaction points. Defined as:

$$TEA = \frac{1}{T} \sum_{t=0}^T \mathbf{1}(\hat{\mathbf{a}}_t \in \mathcal{R}_t) \quad (3)$$

where $\hat{\mathbf{a}}_t$ is the predicted intermediate action state, $\mathbf{a}_t^{gt}$ is the ground-truth intermediate action state, $\mathcal{R}_t$ is the corresponding acceptance region, t is the timestamp, and T is the number of timesteps.

Online Evaluation. Offline evaluation metrics are inherently limited as they solely measure static stepwise trajectory precision, neglecting dynamic closed-loop [45] interactions and UI state changes in real-world GUI drag tasks, which are missing from offline evaluation, or naive endpoint-pair evaluation in existing benchmarks. Notably, online evaluation naturally captures real-world trajectory variance, where agents can successfully achieve task goals despite diverse trajectory patterns.

However, constructing realistic online evaluation environments [29] is non-trivial, such environments should provide diversity across multiple domains, while preserving the reproducibility. One naive approach is to set up the OS and software applications as environment, however, such designs either by manual setup or virtual machine image are costly and difficult to reproduce as the initial state of drag tasks are highly diverse. Therefore, we design a **data-driven** approach to enable policy closed-loop rollouts. For each drag task, we store the video recording, task specification, and dense drag trajectory, providing extensive possible GUI states encountered during dragging. During rollouts, the model’s predicted action is matched to the nearest

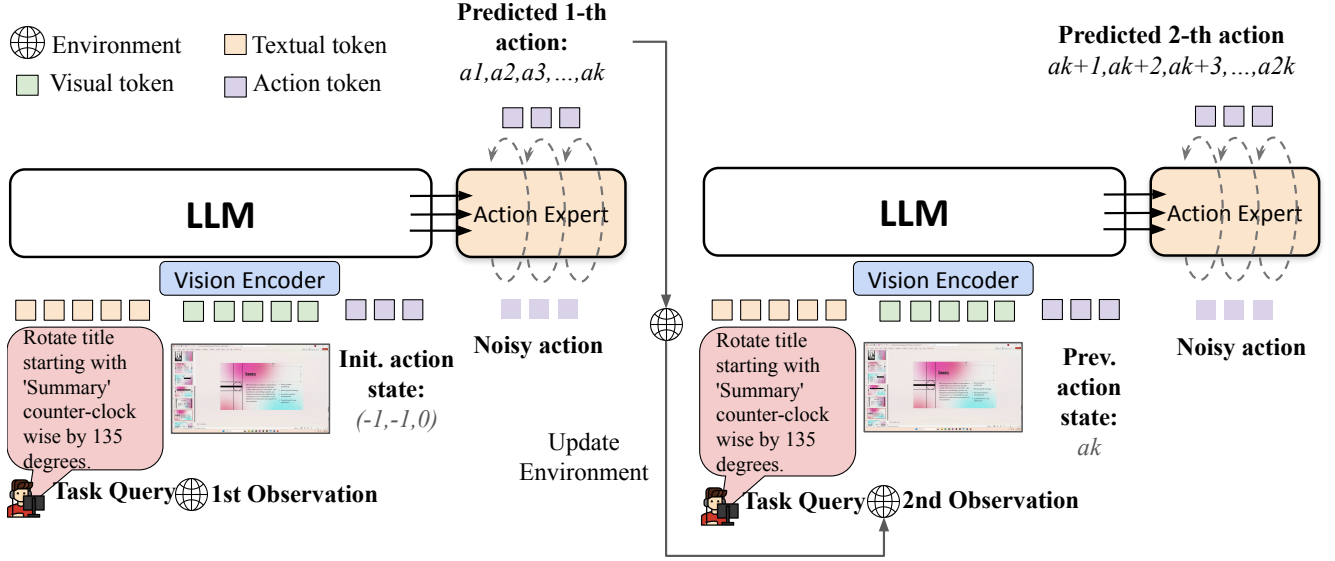


Figure 5. **Overview of ShowUI- π .** Given a task query and visual observations, the model first processes them through the VLM to obtain intermediate hidden states, which are then attended by the action expert. During interaction, the predicted actions update the environment, the next observation is encoded, and a new action chunk is produced—enabling fine-grained, closed-loop cursor control.

recorded state if it falls within a tolerance ϵ (e.g., within 20 pixels of a ground-truth waypoint), upon which the corresponding next observation is retrieved. See the Supp. for more details. (iii) *Task Success Rate*: In this data-driven online environment, the agent receives the current visual observation and task instruction, generates an action, and subsequently obtains an updated visual observation from the approximated corresponding state from the extensive states. We measure performance primarily through binary task success (*success or failure*) per rollout, computed as the number of successful tasks against the total number of tasks.

$$\text{Success Rate} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\hat{\mathbf{a}}_{i,T_i} \in \mathcal{R}_i^{\text{goal}}), \quad (4)$$

where N is the number of online rollouts (tasks), $\hat{\mathbf{a}}_{i,T_i}$ is the final action state of the i -th rollout, and $\mathcal{R}_i^{\text{goal}}$ is the goal acceptance region for that task (e.g., all end states within ϵ pixels of the ground-truth goal). See Supp. for illustrations.

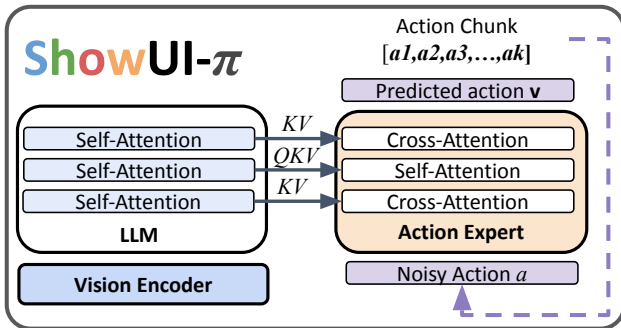


Figure 6. **Architecture of ShowUI- π .** ShowUI- π uses an LLM with cross-attention to a lightweight action expert to generate unified action chunks that handle both discrete clicks and continuous drag segments.

4. ShowUI- π Model

Overview. As outlined in Fig. 6, ShowUI- π is built upon the SmolVLA-450M [39] architecture, with tailored designs for streaming GUI control. ShowUI- π consists of two primary integrated components: (i) a pretrained VLM, initialized by SmolVLM-2 [30] and (ii) a flow matching action expert. The VLM efficiently processes multimodal inputs, encoding screenshot o into visual token representations in a unified embedding space together with projected action states and language instructions $\mathcal{Q}$. The action expert is built on a transformer with the same number of layers as the VLM backbone (i.e. 16). During action prediction, the corresponding layers of the VLM backbone and the action expert perform interleaved self-attention and cross-attention. The *action state* a from the previous step is projected back into the VLM backbone to condition subsequent predictions; it is initialized as $a_0 = [-1, -1]$ and later updated using the last action a_k . The action expert is trained with flow matching, executed k times per trajectory, where each step refines noisy actions into clean predictions, enabling smooth and deterministic trajectory generation. We next illustrate our three designs for effectively modeling: (i) Unified action representation; (ii) Flow-based trajectory generation; (iii) Directional regularization for stable trajectory prediction.

4.1. Unified Continuous and Discrete Actions

Clicks and drags exhibit different temporal and spatial dynamics, making it nontrivial to integrate them within a unified model. Yet such unification is crucial: a single model capable of handling both interaction types can flexibly adapt to diverse GUI tasks that require switching between discrete

and continuous actions. Building upon our trajectory generation, we aim to unify discrete clicks and continuous drags under a unified action modeling framework. Observing that “a click is essentially a drag with negligible movement”, we thus unify click or drag action interactions as sequences $\mathcal{A}$ of (x, y, m) triplets, where (x, y) are cursor coordinates and m is an atomic up and down mouse button state.

$\mathcal{A} = [\mathbf{a}_1, \dots, \mathbf{a}_H]$, $\mathbf{a}_k = (x_k, y_k, m_k)$, $m_k \in \{\text{down}, \text{up}\}$. (i) Within this representation, clicks become minimal two-step trajectories: $[(x_1, y_1, \text{down}), (x_1, y_1, \text{up})]$ while (ii) Drags, become naturally represented as extended incremental press-hold trajectories:

$$[(x_1, y_1, \text{down}), (x_2, y_2, \text{down}), \dots, (x_T, y_T, \text{up})].$$

This unified representation greatly simplifies the action space, removing the need for rigid, predefined action formats used in prior GUI agents (e.g., language tokens). By modeling all interactions as continuous sequences, the policy naturally supports flexible multi-dataset co-training, seamlessly integrating both clicking [7, 26] and dragging within a single unified framework.

4.2. Flow-based Continuous Trajectory

To achieve real-time interactions—which demand smooth and efficient trajectory generation—we adopt a flow-based incremental generation framework [27] driven by a lightweight conditional vector field $\mathbf{v}_\theta$:

$$\frac{d\hat{\mathbf{a}}(s)}{ds} = \mathbf{v}_\theta(\hat{\mathbf{a}}(s), s \mid o_t, \mathcal{Q}), \quad (5)$$

where o_t is the current visual observation, $\mathcal{Q}$ is the task instruction, and $\hat{\mathbf{a}}(s) = [x, y, m]$ represents the predicted action trajectory as in § 4.1. Moreover, $s \in [0, 1]$ is a continuous parameter of the flow, smoothly moving the cursor from the segment’s start where $s = 0$ to its end where $s = 1$.

Instead of treating all trajectory steps equally as in the naive flow-matching loss (Eq. 1), we emphasize the steps that matter most for successful GUI interaction. In particular, early movements must be conditioned on the start point, and the final steps must land precisely on the intended end-point. To capture this, we introduce a *reweighting* scheme that increases the contribution of both the initial and terminal segments of the trajectory:

$$\mathcal{L}_{\text{weighted}} = \sum_{t=1}^T w_t \mathcal{L}_{\text{flow matching}}^{(t)} \quad (6)$$

where w_t is a constant that assigns larger weights to critical steps. We adopt a simple scheme that assigns a weight of 10 to the start and end points, otherwise 1.

4.3. Directional Regularization

GUI actions inherently demand smooth, directionally coherent cursor movements. Standard flow-based methods tend to optimize trajectory magnitude without explicitly enforcing directional consistency. This directional misalignment can cause critical failures in GUI tasks, such as incorrect cursor orientation or jitter. To explicitly regularize

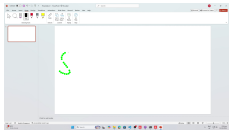
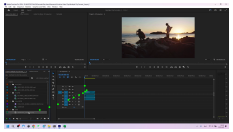
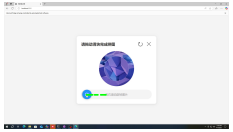
Domain	Task Instruction	Trajectory Visualization
PowerPoint	Rotate center Fox clockwise by 45 degrees.	
Handwriting	Write “Starlit grin” on the canvas	
Adobe Premiere Pro	Apply Unsharp Mask effect to Space clip	
Captcha	Solve the rotate Captcha	

Table 1. **Illustration of ShowUI- π predicted trajectories across four domains.** ShowUI- π generates smooth, human-like trajectories that closely follow the instructed paths.

directional alignment, we introduce a **directional regularization** loss term, defined as

$$\mathcal{L}_{\text{reg}} = \frac{1}{T} \sum_{t=1}^T (1 - \cos(\hat{\mathbf{a}}_t, \mathbf{u}_t)), \quad (7)$$

where $\hat{\mathbf{a}}_t$ and $\mathbf{u}_t$ represent predicted and ground-truth point. **Total Objective.** By combining the reweighting flow-matching objective with directional regularization, the final training objective is defined as:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{weighted}} + \lambda \mathcal{L}_{\text{reg}}, \quad (8)$$

where λ controls the balance between trajectory accuracy and directional consistency. We set λ to 0.1 to ensure comparable magnitudes of loss terms.

5. Experiments

Our experiments systematically investigate the following research questions: **Q1:** How does ShowUI- π compare to existing mainstream GUI agents on performing dexterous operations? **Q2:** What factors contribute to the performance of ShowUI- π ? **Q3:** How well does ShowUI- π generalize to different task domains under multi-dataset training?

Baselines. We compare ShowUI- π with several representative baselines, including both proprietary models and open-source models, including OpenAI Operator [34], Gemini-2.5-CUA [17], Seed 1.6-Vision [5], Qwen3-VL [38], UI-TARS 1.5 [6], OpenCUA [42], etc. Because baseline models output actions as language, we use only their first predicted action for offline evaluation. For online evaluation, they interact with our data-driven closed-loop environment for fair comparison, and a rollout is marked successful if the goal is reached at any step. To control computational cost, we limit each baseline to three interaction steps.

Methods	Offline Endpoint Accuracy (% , $\uparrow$)						Offline Trajectory Error ($\downarrow$)					
	OS	PPT	Premiere	Captcha	Handwriting	Overall	OS	PPT	Premiere	Captcha	Handwriting	Overall
Action as Language — Proprietary models												
Operator	51.49	3.96	0.00	0.00	0.00	11.09	46.85	244.25	602.00	0.00	795.58	422.17
UI-TARS-1.5-7B	60.40	0.99	0.00	3.96	0.00	13.07	224.57	235.50	564.89	172.53	703.54	380.21
Seed-1.6-Vision	71.29	1.98	0.99	4.95	0.00	15.84	62.17	233.92	467.09	81.64	824.55	333.87
Gemini-2.5-CUA	84.16	11.88	0.00	3.96	0.00	20.00	12.11	121.94	508.19	114.34	0.00	189.15
Action as Language — Open-source models												
OpenCUA-32B	96.04	6.93	0.00	0.00	0.00	20.59	10.95	165.70	907.41	62.06	791.88	387.60
OpenCUA-7B	99.01	4.95	0.00	3.96	0.00	21.58	4.42	186.65	762.77	214.81	959.10	425.55
Qwen3-VL-32B	71.29	1.98	0.99	0.00	0.00	14.85	149.77	212.02	527.23	236.83	646.73	354.52
Qwen3-VL-8B	8.91	1.98	0.00	0.00	0.00	2.18	660.33	228.47	532.71	257.44	662.37	468.26
Qwen3-VL-2B	3.96	0.00	0.99	0.00	0.00	0.99	708.68	248.29	551.15	258.07	562.62	465.76
Flow-based action (Ours)												
ShowUI-π-450M	61.39	85.06	56.92	96.30	93.07	78.55	350.55	57.68	195.96	136.47	54.57	159.05

Table 2. Main Results on ScreenDrag offline evaluation. Left: Endpoint Accuracy; Right: Trajectory Error.

Trajectory modeling	Offline Evaluation. Avg. Endpoint Accuracy ($\uparrow$) / Trajectory Error ($\downarrow$).					
	OS	PowerPoint	Premiere	Captcha	Handwriting	Overall
SmolVLM	1.98 / 566.98	0.00 / 305.64	0.00 / 493.34	0.00 / 254.14	0.00 / 440.42	0.40 / 412.10
Diffusion Policy	43.47 / 553.66	45.17 / 154.11	29.70 / 347.88	64.44 / 203.46	53.56 / 80.49	47.33 / 267.92
Flow Matching (Our)	61.39 / 350.55	85.06 / 57.68	56.92 / 195.96	96.30 / 136.47	93.07 / 54.57	78.55 / 159.05

Table 3. Ablation of different modeling approaches under the same SmolVLM backbone. We compare language modeling, diffusion policy, and flow matching, all trained on the same 20K training trajectories. Our flow-matching delivers the most effective performance.

Methods	Online Success Rate (% , $\uparrow$)					
	OS	PPT	Premiere	Captcha	Handwriting	Overall
Action as Language — Proprietary models						
Operator	53.47	9.90	2.97	0.00	0.00	13.27
UI-TARS-1.5-7B	73.27	1.98	1.98	7.92	0.00	17.03
Seed-1.6-Vision	77.23	3.96	1.98	8.91	2.97	19.01
Gemini-2.5-CUA	86.14	20.79	0.00	3.96	0.00	22.18
Action as Language — Open-source models						
OpenCUA-32B	97.03	6.93	0.00	0.00	0.00	20.79
OpenCUA-7B	99.01	4.95	0.00	5.94	0.00	21.98
Qwen3-VL-32B	83.17	4.95	3.96	2.97	0.00	19.01
Qwen3-VL-8B	24.75	5.94	6.93	0.00	0.00	7.52
Qwen3-VL-2B	13.86	6.93	2.97	0.00	0.00	4.75
Flow-based action (ours)						
ShowUI-π-450M	13.11	22.93	8.64	55.91	34.32	26.98

Table 4. Main Results on ScreenDrag (Online Success Rate).

5.1. Main Results

Online Evaluation. In Tab. 4, ShowUI- π attains an overall online closed-loop success rate of 26.98%, surpassing the SOTA proprietary model Gemini-2.5-CUA by 4.8%, and the SOTA open-source model OpenCUA-7B by 6.19%. It is worth noting that ShowUI- π performs well on tasks involving free-form drag actions, *e.g.*, PowerPoint where circular drag is required to rotate elements, and Handwriting where trajectories are non-linear. Moreover, ShowUI- π performs well on Captcha tasks where observation on-the-fly is required. Interestingly, Operator failed all Captcha tasks as it refused to solve Captcha due to its safety policy, and Gemini-2.5-CUA always mistakenly calls open-browser tool when executing handwriting tasks. Furthermore, OpenCUA-7B reaches the second highest perfor-

w	Online Success Rate (% , $\uparrow$)					
	OS	PPT	Premiere	Captcha	Handwriting	Overall
1	12.87	11.49	10.77	7.41	9.90	10.49
5	11.69	17.30	7.84	19.40	16.22	14.49
10	13.11	22.93	8.64	55.91	34.32	26.98
15	26.73	11.49	7.69	33.33	24.75	20.80

Table 5. Effects of Temporal Weights. Removing temporal weights reduces online success across domains.

mance among all baseline models, demonstrating that increasing parameter size cannot always improve the action performance, *i.e.*, drags and clicks, of computer use agents, and lightweight models such as OpenCUA-7B and ShowUI- π are worth further development.

Offline Evaluation. As shown in Tab. 2, endpoint accuracy and trajectory error are computed in offline evaluation to evaluate whether the trajectory closely aligns with the ground-truth trajectory. For baseline models which output actions as language, only the endpoints are used to compute the trajectory error. For ShowUI- π , all waypoints are used to evaluate whether the model learned to closely follow the trajectory. It can be observed from Tab. 2 that ShowUI- π produces the least trajectory error, showing that the flow-based VLA can follow the learned trajectory pattern well.

5.2. Key Ablations

Which action modeling approach is most effective? Flow matching delivers the highest endpoint accuracy of 78.55% and lowest trajectory error of 159.05 px, surpassing diffu-

Chunk Size length($\hat{a}$)	Exec. Steps before next observation	Offline eval.	
		Endpoint Acc. ($\uparrow$)	Traj. Error ($\downarrow$)
10	1	75.45	176.43
	2	72.28	193.81
	5	66.34	241.60
20	1	78.55	159.05
	2	76.35	177.87
	5	79.22	205.23

Table 6. **Ablation on Chunk Size and Execution Steps.** Two chunk sizes ($n \in \{10, 20\}$); each evaluated at execution steps $\{1, 2, 5\}$. We report offline Endpoint Acc. and Traj. Error.

Method	#Params	Drag	
		Online SR.	Offline Acc.
Separate Heads	550M	23.25	79.22
Unified Head	450M	26.98	78.55

Table 7. **Unified vs. Separate Action Heads.** Unified head provides comparable performance to separate heads, while being more elegant and practical.

sion policy by 31.22% and lower error, and language modeling by 78.15% and much lower error as shown in Tab. 3. This demonstrates flow matching’s deterministic velocity field effectively captures complex, free-form GUI drags with its training stability.

How do chunk size and execution steps impact model performance? As shown in Tab. 6, increasing chunk size, *i.e.*, length of steps in each prediction, from 10 to 20 generally boosts the performance, indicating predicting more action steps is helpful for learning to capture the action distribution. Interestingly, when the execution steps are large, *i.e.*, executing more steps from prediction before the next re-observation, increasing chunk size can help alleviate the degradation. When the execution steps are 5, increasing chunk size from 10 to 20 can improve its accuracy by 12.9% and reduce trajectory error by 36.4 px. Moreover, fewer execution steps generally produce the highest accuracy and least error, showing that frequent re-observation enhances precision. Therefore, chunk size 20 with execution step 1 achieves the tradeoff for reliable and precise actions.

How does applying reweighting affect model performance? As shown in Eq. 6, reweighting applies step-dependent coefficients to the flow matching loss across the predicted action horizon, prioritizing accuracy at important steps, *i.e.*, after drag starts and near drag ends. Reweighting at scale 10 yields the highest overall success rate, increasing from 10.49% without weighting to 26.98%, with dramatic gains on Captcha by 48.50%, showcasing the importance of starting and ending steps for successful drags. Interestingly, overweighing, *i.e.*, with scale 15 benefits OS File drag but reduces overall performance, showing that overweighing may harm learning of intermediate actions, and moderate weighting optimally learns the entire drag, shown in Tab. 5.

Unified action head or separate heads for drags and

Directional Reg.	OS	PPT	Online Success Rate (% , $\uparrow$)			
			Premiere	Captcha	Handwriting	Overall
w/o. ($\lambda = 0$)	10.83	14.37	8.26	14.92	14.78	12.63
w. ($\lambda = 0.1$)	13.11	22.93	8.64	55.91	34.32	26.98

Table 8. **Effects of Directional Regularization.** Directional regularization improves online success across all domains, especially in domains that are sensitive to drag direction, *e.g.*, Captcha.

clicks? As shown in Tab. 7, the unified head matches separate heads in offline accuracy at 78.55% versus 79.22%, while improving online drag success by 3.7% and reducing model size by 100M parameters. Though the two designs have comparable performance, the unified design is more elegant, and allows ShowUI- π to handle both drag and click tasks without task-dependent head selection. Conversely, the separate head design introduces impractical head selection and requires an additional 100M parameters, resulting in higher computational overhead.

Is Directional Regularization helpful? It can be observed from Tab. 8 shows that directional regularization consistently improves ShowUI- π across all domains, as GUI drag tasks demand smooth and directionally correct trajectories, *e.g.*, Captcha-solving may fail to place the slider or rotate to the wrong angle if the drag direction is inaccurate.

5.3. Qualitative Analysis

In Fig. 1, we demonstrate how ShowUI- π produces precise trajectory generation aligned with task intent across different domains. In PowerPoint, the model accurately performs the required 45-degree clockwise rotation, producing a trajectory consistent with the instructed geometric transform. In Handwriting, it generates a smooth, continuous stroke that clearly forms the intended “S” shaped curve in “Starlit grin.” For Captcha, the model exhibits fine-grained control, stopping at the exact rotation angle needed to solve the puzzle rather than overshooting. These results collectively show that ShowUI- π can follow continuous, high-degree-of-freedom motions with instruction-following.

6. Conclusion

We introduce ShowUI- π , the first flow-based generative GUI agent designed for continuous trajectory control. ShowUI- π enables precise, real-time cursor adjustment. To support evaluation, we release ScreenDrag, a benchmark covering five domains with both offline and online rollout protocols. Extensive experiments show that ShowUI- π sets a leading results with only 450M parameters. We hope this work lays the foundation for building GUI agents with truly human-like dexterity.

Limitations and Future works. We trained ShowUI- π at a small model size and limited training data scale. In our future work, we will scale up the model size.

Acknowledgements

This research is supported by the National Research Foundation Singapore under its AI Singapore Programme (Award Number: AISG3-RP-2022-030).

References

- [1] Anthropic. Claude sonnet 4.5 system card, 2025. Accessed: 2025-11-13.
- [2] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, et al. Paligemma: A versatile 3b vlm for transfer. *arXiv preprint arXiv:2407.07726*, 2024.
- [3] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A Vision–Language–Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164*, 2024.
- [4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [5] ByteDance Seed. Seed1.6, 2025. Accessed: 2025-11-11.
- [6] ByteDance Seed. Ui-tars, 2025. Accessed: 2025-11-11.
- [7] Wentong Chen, Junbo Cui, Jinyi Hu, Yujia Qin, Junjie Fang, Yue Zhao, Chongyi Wang, Jun Liu, Guirong Chen, Yupeng Huo, et al. Guicourse: From general vision language models to versatile gui agents. *arXiv preprint arXiv:2406.11317*, 2024.
- [8] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing gui grounding for advanced visual gui agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9313–9332, 2024.
- [9] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023.
- [10] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114, 2023.
- [11] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, et al. Workarena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718*, 2024.
- [12] Aarash Feizi, Shravan Nayak, Xiangru Jian, Kevin Qinghong Lin, Kaixin Li, Rabiul Awal, Xing Han Lù, Johan Obando-Ceron, Juan A Rodriguez, Nicolas Chapados, et al. Grounding computer use agents on human demonstrations. *arXiv preprint arXiv:2511.07332*, 2025.
- [13] Hiroki Furuta, Ofir Nachum, Kuang-Huei Lee, Yutaka Matsuo, Shixiang Shane Gu, and Izzeddin Gur. Multimodal web navigation with instruction-finetuned foundation models. *arXiv preprint arXiv:2305.11854*, 2023.
- [14] Difei Gao, Lei Ji, Zechen Bai, Mingyu Ouyang, Peiran Li, Dongxing Mao, Qinchun Wu, Weichen Zhang, Peiyi Wang, Xiangwu Guo, et al. Assistgui: Task-oriented desktop graphical user interface automation. *arXiv preprint arXiv:2312.13108*, 2023.
- [15] Difei Gao, Lei Ji, Luowei Zhou, Kevin Qinghong Lin, Joya Chen, Zihan Fan, and Mike Zheng Shou. Assistgpt: A general multi-modal assistant that can plan, execute, inspect, and learn. *arXiv preprint arXiv:2306.08640*, 2023.
- [16] Difei Gao, Siyuan Hu, Zechen Bai, Qinghong Lin, and Mike Zheng Shou. Assisteditor: Multi-agent collaboration for gui workflow automation in video creation. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 11255–11257, 2024.
- [17] Google AI. Gemini 2.5 computer use (cua), 2025. Accessed: 2025-11-11.
- [18] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for gui agents. *arXiv preprint arXiv:2410.05243*, 2024.
- [19] Izzeddin Gur, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. A real-world webagent with planning, long context understanding, and program synthesis. *arXiv preprint arXiv:2307.12856*, 2023.
- [20] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- [21] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 14281–14290, 2024.
- [22] Siyuan Hu, Mingyu Ouyang, Difei Gao, and Mike Zheng Shou. The dawn of gui agent: A preliminary case study with claude 3.5 computer use. *arXiv preprint arXiv:2411.10323*, 2024.
- [23] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [24] Zeyi Liao, Yadong Lu, Boyu Gou, Huan Sun, and Ahmed Awadallah. Beyond clicking: A step towards generalist gui grounding via text dragging. *arXiv preprint arXiv:2601.06031*, 2025.
- [25] Kevin Qinghong Lin, Linjie Li, Difei Gao, Qinchun Wu, Mingyi Yan, Zhengyuan Yang, Lijuan Wang, and Mike Zheng Shou. Videogui: A benchmark for gui automation from instructional videos. In *NeurIPS*, 2024.
- [26] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Zechen Bai, Weixian Lei, Lijuan Wang, and Mike Zheng Shou. Showui: One vision-language-action model for generalist gui agent. In *NeurIPS 2024 Workshop on Open-World Agents*, 2024.
- [27] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

- [28] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.
- [29] Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. *arXiv preprint arXiv:2108.03298*, 2021.
- [30] Andrés Marafioti, Orr Zohar, Miquel Farré, Merve Noyan, Elie Bakouch, Pedro Cuenca, Cyril Zakka, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, et al. Smolvlm: Redefining small and efficient multimodal models. *arXiv preprint arXiv:2504.05299*, 2025.
- [31] Shravan Nayak, Xiangru Jian, Kevin Qinghong Lin, Juan A Rodriguez, Montek Kalsi, Rabiul Awal, Nicolas Chapados, M Tamer Özsu, Aishwarya Agrawal, David Vazquez, et al. Ui-vision: A desktop-centric gui benchmark for visual perception and interaction. *arXiv preprint arXiv:2503.15661*, 2025.
- [32] Dantong Niu, Yuvan Sharma, Giscard Biamby, Jerome Quenum, Yutong Bai, Baifeng Shi, Trevor Darrell, and Roei Herzig. Llarva: Vision-action instruction tuning enhances robot learning. *arXiv preprint arXiv:2406.11815*, 2024.
- [33] OpenAI. Gpt-5 system card, 2025. Accessed: 2025-11-13.
- [34] OpenAI. Operator, 2025. Accessed: 2025-11-11.
- [35] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- [36] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: A Vision–Language–Action Model with Open-World Generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [37] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025.
- [38] Qwen Team. Qwen3-vl, 2025. Accessed: 2025-11-11.
- [39] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- [40] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pages 2256–2265. pmlr, 2015.
- [41] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution, 2024.
- [42] Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, et al. Opencua: Open foundations for computer-use agents. *arXiv preprint arXiv:2508.09123*, 2025.
- [43] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.
- [44] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025.
- [45] Norbert Wiener. *Cybernetics: or Control and Communication in the Animal and the Machine*. The Technology Press, Cambridge, MA, 1948.
- [46] Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguvis: Unified pure vision agents for autonomous gui interaction. *arXiv preprint arXiv:2412.04454*, 2024.
- [47] Jianwei Yang, Reuben Tan, Qianhui Wu, Ruijie Zheng, Baolin Peng, Yongyuan Liang, Yu Gu, Mu Cai, Seonghyeon Ye, Joel Jang, et al. Magma: A foundation model for multimodal ai agents. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 14203–14214, 2025.
- [48] Rui Yang, Lin Song, Yanwei Li, Sijie Zhao, Yixiao Ge, Xiu Li, and Ying Shan. Gpt4tools: Teaching large language model to use tools via self-instruction. *Advances in Neural Information Processing Systems*, 36:71995–72007, 2023.
- [49] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022.
- [50] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- [51] Keen You, Haotian Zhang, Eldon Schoop, Floris Weers, Amanda Swearngin, Jeffrey Nichols, Yinfei Yang, and Zhe Gan. Ferret-ui: Grounded mobile ui understanding with multimodal llms. In *European Conference on Computer Vision*, pages 240–255. Springer, 2024.
- [52] Andy Zeng, Maria Attarian, Brian Ichter, Krzysztof Choromanski, Adrian Wong, Stefan Welker, Federico Tombari, Aveek Purohit, Michael Ryoo, Vikas Sindhwani, et al. Socratic models: Composing zero-shot multimodal reasoning with language. *arXiv preprint arXiv:2204.00598*, 2022.
- [53] Chaoyun Zhang, Liqun Li, Shilin He, Xu Zhang, Bo Qiao, Si Qin, Minghua Ma, Yu Kang, Qingwei Lin, Saravan Rajmohan, et al. Ufo: A ui-focused agent for windows os interaction. In *Proceedings of the 2025 Conference of the Nations*

of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pages 597–622, 2025.

- [54] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. Gpt-4v (ision) is a generalist web agent, if grounded. *arXiv preprint arXiv:2401.01614*, 2024.
- [55] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.
- [56] Minjie Zhu, Yichen Zhu, Jinming Li, Junjie Wen, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, Yaxin Peng, Feifei Feng, et al. Scaling diffusion policy in transformer to 1 billion parameters for robotic manipulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 10838–10845. IEEE, 2025.
- [57] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.